

République Tunisienne
École supérieure privée d'ingénierie et de technologie

Rapport de Stage d'Immersion en Entreprise

Sujet :
**Conception et évaluation d'un système de
question-réponse documentaire multilingue (RAG)
appliqué à des documents bancaires**

Réalisé au sein de
Al Baraka Bank

Du 1er août au 1er octobre 2026

Élaboré par TODO-NOM Prénom
Encadré par Ahlem BENHADDOUD

Année universitaire 2025 - 2026

Table des matières

Dédicaces

Remerciements

Introduction Générale

Chapitre 1 — Présentation de l'organisme d'accueil et étude préalable

Chapitre 2 — Conception de la solution

Chapitre 3 — Réalisation, traitements et résultats

Chapitre 4 — Conclusion générale et perspectives

Note : le champ TODO-NOM de la page de garde reste à remplir (voir les `\newcommand` en tête de `rapport_stage.tex`). Tous les chiffres du rapport sont issus des mesures du stage (harnais, bancs d'essai et runs CI de septembre 2026).

Dédicaces

J'ai le plaisir de dédier ce travail à :

- Mes très chers parents, pour leur amour, leur patience et leurs encouragements sans faille.
- Mes sœurs et mon frère, pour leur soutien et leur présence rassurante.
- Mes amis proches qui ont toujours été là pour me soutenir dans ce parcours.
- Mes grands-parents, dont les prières m'accompagnent chaque jour.
- Mes oncles et tantes qui m'ont toujours soutenue, chacun à sa manière.
- Tous mes collaborateurs et collègues de stage pour leur soutien technique et moral.

Je vous dédie cet humble travail, fruit d'un long parcours.

Remerciements

Au terme de ce travail, je tiens à remercier toutes les personnes qui ont contribué à la réalisation de ce projet.

- Je remercie sincèrement Ahlem BENHADDOD, mon encadrant, pour son accompagnement, ses conseils précieux et sa disponibilité tout au long du stage.
- Mes parents pour leur soutien constant.
- Mes sœurs et mon frère pour leur présence rassurante et leurs encouragements.
- Mes amis proches, qui ont toujours été là pour moi.
- Tous mes collaborateurs qui m'ont soutenue tout au long de la réalisation du projet, avec leur soutien moral et technique, leur bonne humeur et leur sympathie.
- Tous les enseignants qui ont participé à mon évolution scientifique durant les années précédentes.
- Toutes les personnes qui ont contribué à l'élaboration de ce travail. Il n'est malheureusement pas possible de les citer toutes ici, mais elles se reconnaîtront : je leur adresse mes plus profonds remerciements.

Introduction Générale

Les grands modèles de langage (LLM) rédigent des réponses fluides, mais ils ont un défaut majeur pour un usage professionnel : ils peuvent affirmer avec assurance des faits faux ou invérifiables. Dans un domaine réglementé comme la banque, une réponse sans source exploitable est inutilisable. La génération augmentée par récupération (RAG, Retrieval-Augmented Generation) apporte une réponse à ce problème : le modèle ne répond qu'à partir de passages réellement extraits d'un corpus approuvé, et chaque affirmation est rattachée à une citation vérifiable.

Ce rapport présente le travail mené au sein de l'entreprise Al Baraka Bank, dont l'objectif est la conception et l'évaluation d'un système RAG multilingue (arabe, français, anglais) appliqué à des documents bancaires : lois, circulaires de la Banque Centrale de Tunisie et guides internes. Le système, nommé RAGLab, couvre toute la chaîne : ingestion de documents PDF/DOCX, découpage sémantique, indexation vectorielle, recherche cross-lingue et génération de réponses citées avec refus contrôlé.

Cette étude inclut également la comparaison mesurée de stratégies de découpage, le banc d'essai de modèles de réponse gratuits, un juge de retrieval sans LLM et l'industrialisation du service (microservice REST, image Docker portable, intégration continue). Le système livré pendant le stage représente environ 66 000 lignes de code, de mesures et de documentation, dont 39 modules Python (environ 21 300 lignes), 6 jeux de questions et 7 pipelines d'intégration continue).

Chapitre 1

Présentation de l'organisme d'accueil et étude préalable

1.1 Introduction

L'entreprise Al Baraka Bank est la première banque islamique de Tunisie et du Maghreb. Elle offre une gamme de produits et de services conformes aux principes de la finance islamique, et investit dans l'intelligence artificielle appliquée à la recherche d'information pour mieux exploiter sa documentation réglementaire et métier.

1.2 Présentation de l'organisme

Al Baraka Bank

Créée le 15 juin 1983 sous la dénomination Beit Ettamwil Saoudi Tounsi (Best Bank), Al Baraka Bank a pris son nom actuel en 2009 dans le cadre de l'unification de l'identité du groupe. C'est une filiale du groupe bancaire international Al Baraka Banking Group (siège à Bahreïn), acteur de référence de la finance islamique. Son siège social est à Tunis et son capital s'élève à 120 millions de dinars.

Technologies mobilisées pour le projet

Étape	Composant	Technologies principales
Ingestion	Parsing PDF/DOCX, normalisation	Python, pypdf, XML natif
Découpage	Chunking sémantique	tiktoken cl100k_base, règles métier
Indexation	Recherche vectorielle + lexicale	ChromaDB (cosinus), BM25
Réponse	Génération citée	API NVIDIA, passerelle xKiro (Qwen)
Service	API REST conteneurisée	FastAPI, Uvicorn, Docker
Qualité	Tests et mesures	unittest, intégration continue (7 pipelines)

Table 1 — Briques technologiques du laboratoire RAGLab.

1.3 Étude de l'existant

Description de l'existant

Avant le début du projet, la documentation bancaire de référence (textes de loi, circulaires, guides internes) existait sous forme de fichiers dispersés, principalement en arabe et en français. Toute question nécessitait une lecture manuelle par un expert métier : la recherche par mots-clés ne franchit pas la barrière de la langue et les réponses produites n'étaient rattachées à aucune citation vérifiable automatiquement.

Workflow des données avant projet

Flux de traitement documentaire avant le projet

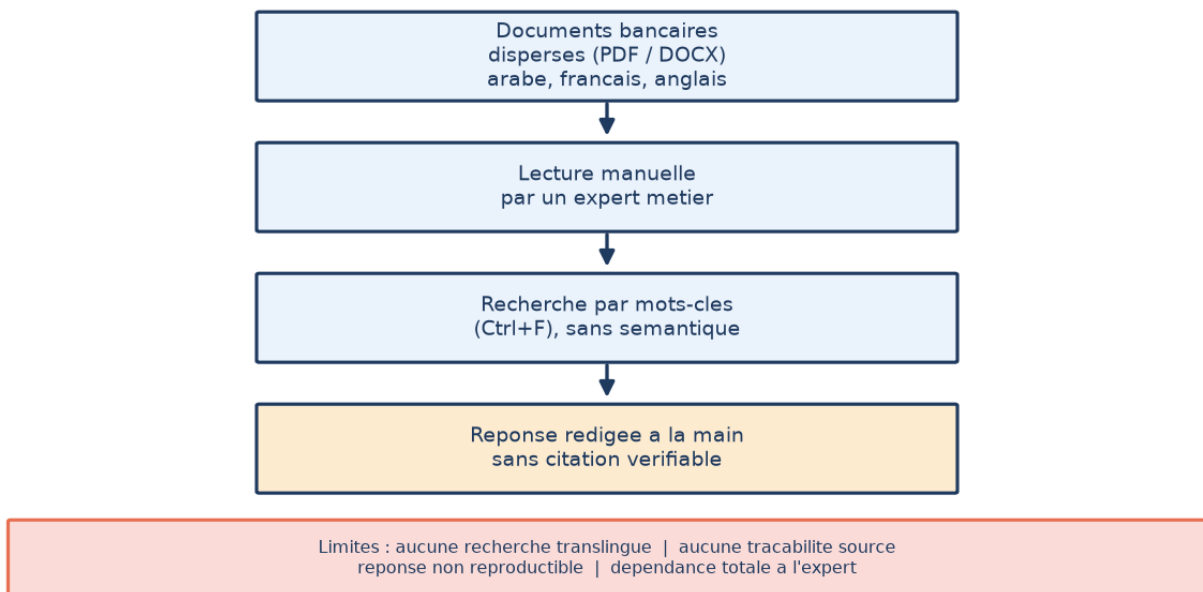


Figure 1 — Flux documentaire avant le projet RAGLab.

Interprétation : les documents étaient consultés manuellement, sans recherche sémantique ni traçabilité — dépendance à l'expert, absence de réponses translingues et de citations contrôlables.

Organisation et livrables du stage

Le stage a produit un laboratoire RAG complet et documenté : 39 modules Python (environ 21 300 lignes), 4 documents bancaires réels d'évaluation, 6 jeux de questions de référence, 7 pipelines d'intégration continue et une image

Docker de production. Ce contenu, et les rapports de mesure qu'il contient, sert de matière première aux chapitres 2 et 3.

Planning prévisionnel du stage

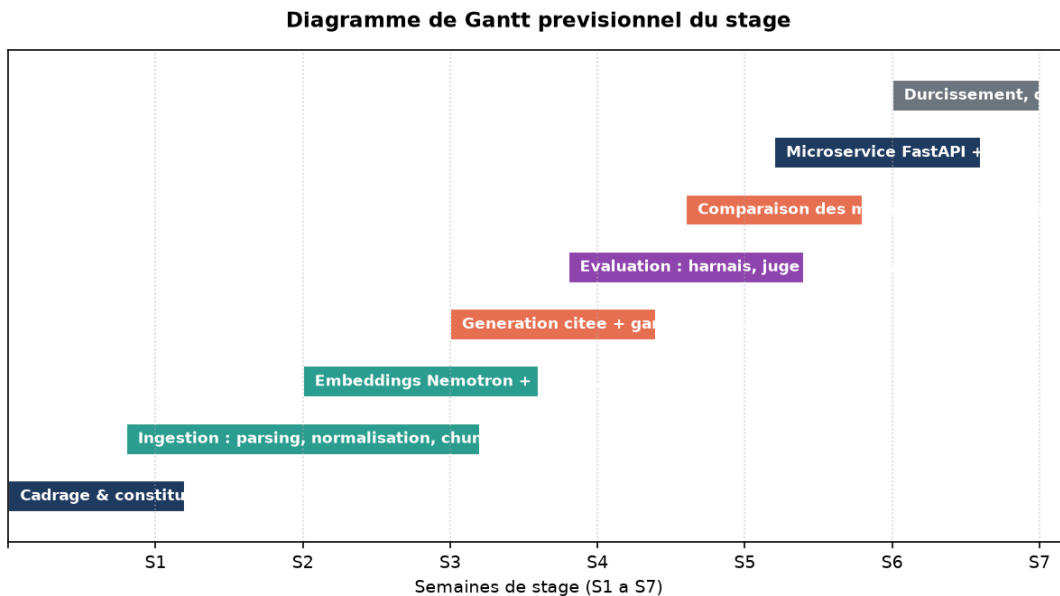


Figure 2 — Diagramme de Gantt du stage (7 semaines, S1 à S7).

Interprétation : planification des activités — corpus, ingestion, embeddings, génération citée, évaluation, comparaison de modèles et industrialisation, du cadrage à la livraison du service conteneurisé.

1.4 Méthodologie adoptée

1. Cadrage et corpus : sélection de documents bancaires réels, avec jeux de questions gelés par langue.
2. Ingestion reproductible : parsing, normalisation arabe, découpage versionné (empreinte v4) et cache d'embeddings resumable.
3. Recherche contrainte et génération citée : top-5 cosinus sur requête d'origine, claims JSON à citations mot-à-mot, refus en cas d'échec.
4. Mesure systématique : harnais A/B, juge de retrieval sans LLM, comparaison de modèles gratuits sur rubriques gelées, CI qui rejoue les mesures.
5. Industrialisation : microservice REST, image Docker portable, documentation de contrat pour les équipes clientes.

1.5 Conclusion

Cette première étude permet de cerner les besoins de l'entreprise, de comprendre la structure du corpus bancaire trilingue et de définir les objectifs

du projet : une réponse citée et refusable plutôt qu'une réponse fluide mais invérifiable, tout en identifiant les contraintes (qualité du découpage, coût des appels modèles, limites du gratuit).

Chapitre 2

Conception de la solution

2.1 Introduction

La conception repose sur la définition des besoins, l'architecture fonctionnelle et le choix du pipeline RAG. L'accent a été mis sur la reproductibilité (graines, caches, empreintes), la mesurabilité (jeux gelés, harnais rejouables) et la sécurité des réponses (citations exigées, refus local).

2.2 Environnement de développement

- Python 3.11, sans framework d'orchestration (ni LangChain ni LlamaIndex) et sans SDK propriétaire (HTTPS standard).
- ChromaDB persistant local (similarité cosinus) + BM25 : recherche vectorielle et lexicale sans base externe.
- FastAPI + Uvicorn : microservice REST de 25 routes, documentation OpenAPI à /docs.
- tiktoken (cl100k_base) et pypdf : tokenisation de référence et lecture des PDF.
- Intégration continue : 7 pipelines (tests, images Docker, modèles gratuits, harnais d'évaluation, juge retrieval, test réel, catalogues).

Choix technologiques

Besoin	Choix retenu	Justification
Embeddings	nvidia/nemotron-3-embed-1b, natif 2048 d	Espace unique et stable ; le juge retrieval mesure 62,8 % sur requêtes sans mot commun contre 0 % au lexical.
Réponses	qwen/qwen3.8-max:free via xKiro	Meilleur score mesuré (13/14 dev, 18/18 held-out) avec contrôle de gratuité en direct avant chaque appel.
Retrieval	Top-5 cosinus, requête d'origine	La traduction de requête n'améliorait pas toutes les questions et dégradait le top-1 held-out.

Cadre logiciel	Aucun framework RAG	Dépendances minimales, comportement explicite, image Docker légère et auditable.
----------------	---------------------	--

Table 2 — Justification des choix technologiques.

2.3 Besoins fonctionnels et non fonctionnels

Besoins fonctionnels

- Ingérer des documents PDF, DOCX, TXT et Markdown avec normalisation multilingue.
- Indexer le corpus en chunks adressables, avec une collection par couple (espace d'embedding, mode de chunking).
- Rechercher les passages pertinents pour une question en arabe, français ou anglais, y compris translingue.
- Générer des réponses citées ou refuser explicitement (corpus silencieux, données privées, données temps réel).
- Exposer toutes les fonctions par CLI, console interactive et API REST documentée.

Besoins non fonctionnels

- Traçabilité : chaque affirmation livrée porte des citations mot-à-mot extraites des chunks ; un nombre non cité entraîne un refus.
- Reproductibilité : tokenizer, caches, empreintes et jeux gelés garantissent des mesures rejouables en CI.
- Sécurité : pas de réponse sur données privées ou temps réel (garde-fous locaux), masquage des clés, validation des vecteurs.
- Portabilité : image Docker multi-stage exécutable sans clone ni pip, y compris hors-ligne après import.
- Maintenabilité : un noyau partagé par les trois frontaux, 133 contrôles hors-ligne, documentation de contrat .

2.4 Architecture fonctionnelle

L'architecture se compose de quatre éléments principaux :

Données en entrée	Traitement	Modèles	Résultats
-------------------	------------	---------	-----------

Documents bancaires ar/fr/en	Parsing, normalisation, chunking, embeddings	Nemotron (recherche) + Qwen (rédaction)	Réponses citées ou refus motivés
------------------------------	--	---	----------------------------------

Table 3 — Les quatre blocs de l'architecture.

Description des composants

1. Collecte : acquisition des documents (dépôts le corpus, téléversement POST /documents avec versions et purge ciblée).
2. Transformation : nettoyage, réparation du texte arabe, découpage (restructure/size/manual) et plongements vectoriels.
3. Application : recherche top-5 et exécution du modèle de réponse derrière le portillon de citations.
4. Génération : production de la réponse finale citée, ou d'un refus diagnostiquable avec les extraits fournis.

2.5 Pipeline retenu : retrieval Nemotron + réponses Qwen

Description et justification

Le pipeline est gelé par le module `pipeline_policy.py` : toute substitution de modèle sur les chemins mesurés lève une erreur au lieu d'un appel silencieux vers un autre modèle. Le tableau 4 résume ses paramètres.

Paramètre	Description
<code>embedding_model</code>	nvidia/nemotron-3-embed-1b, natif 2048 dimensions
<code>answer_model</code>	qwen/qwen3.8-max:free via xKiro, prompt grounded-v1
<code>top_k</code>	5 chunks (requête d'origine, similarité cosinus)
<code>chunking</code>	restructure par défaut ; 220/40, tokenizer cl100k_base
<code>batch_size</code>	32 textes par appel d'embeddings, retries + bisection
<code>citation_gate</code>	Citations mot-à-mot exigées, nombres présents dans les citations
<code>free_price_check</code>	Vérification de gratuité en direct avant chaque appel Qwen

Table 4 — Paramètres principaux du pipeline RAG.

2.6 Diagramme de l'architecture

Architecture generale de la solution RAGLab

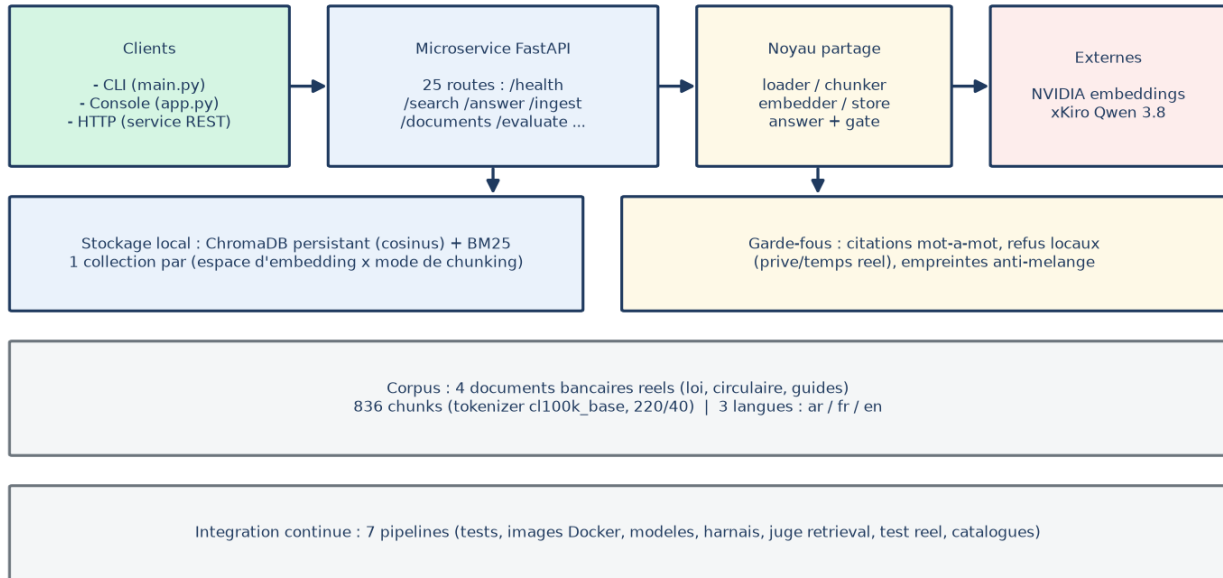
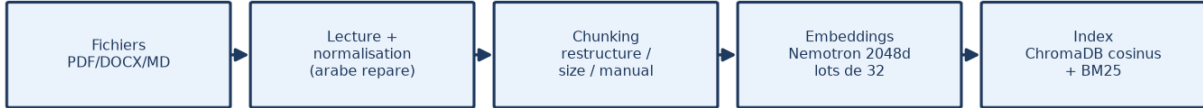
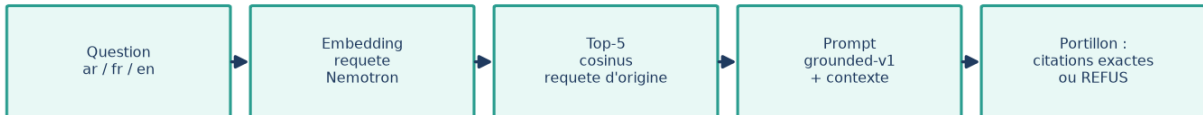


Figure 3 — Architecture générale de la solution.

Interprétation : les trois frontaux partagent un même noyau ; le service REST expose 25 routes ; le stockage et les garde-fous restent locaux ; seuls les embeddings et la rédaction font appel à des modèles externes.

Pipeline de traitement des données

Pipeline de traitement : ingestion et interrogation**INGESTION (hors-ligne)****REQUETE (en ligne)**

Reponse livree = claims JSON portant des citations mot-a-mot extraites des chunks ; tout nombre doit apparaitre dans ses citations, sinon refus structure.

Contrats d'identite : empreinte de chunking + empreinte d'espace d'embedding ; un index obsolete est refuse (re-ingest --reset exige).

Figure 4 — Pipeline d'ingestion et d'interrogation.

Interprétation : l'ingestion (hors-ligne) construit un index décrit par ses empreintes ; la requête (en ligne) suit un chemin contraint qui se termine par le portillon de citations — réponse citée ou refus.

Champs d'un chunk et métadonnées d'index

Champ	Description
index / source	Position du chunk et document d'origine (id source::chunk_NNNN)
heading / language	Titre de section et langue détectée (ar/fr/en)
token_count	Taille mesurée au tokenizer cl100k_base
chunk_fp	Empreinte du découpage (mode, tailles, tokenizer) — v4
embedding_fp	Empreinte de l'espace d'embeddings (modèle, tâche, dimensions)

Table 5 — Champs indexés pour chaque chunk.

2.7 Conclusion

La conception fige un pipeline étroit mais justifié par la mesure : un seul modèle d'embeddings, un seul modèle de réponses, une retrieval top-5 sur requête d'origine et un portillon de citations qui interdit l'affirmation non sourcée. Le chapitre suivant décrit la réalisation et les résultats obtenus.

Chapitre 3

Réalisation, traitements et résultats

3.1 Chargement et exploration du corpus

```
python main.py inspect --data-dir ./corpus
python main.py ingest --reset --data-dir ./corpus
python main.py query "What is Murabaha?" --query-lang en
python main.py answer "What is Murabaha?" --query-lang en
```

Document	Format	Langue	Rôle
Loi_2016-48	PDF (36 p. au total)	arabe	Corpus réel d'évaluation
Circulaire_BCT_2019-08	PDF	arabe	Corpus réel d'évaluation
Guide_Interne_Operations_Bancaires_Islamiques	DOCX	arabe	Corpus réel d'évaluation
Madkhal_Sayrafa_Islamiya	DOCX	arabe	Corpus réel d'évaluation

Table 6 — Composition du corpus.

Métrique	Valeur
Chunks (220/40, cl100k_base)	836
Pages PDF	36
Empreinte du manifeste	807785db... (SHA-256)
Couverture des preuves attendues	16/16 (100 %)
Chunks de métadonnées de titres	0
Dimensions d'embeddings	2048 (natives)

Table 7 — Statistiques descriptives du corpus indexé.

3.2 Analyse détaillée et diagnostics

Couverture des preuves et qualité du découpage

Diagnostic	Résultat
Phrases-preuves joignables dans un chunk	16/16 — tout est atteignable
Chunks parasites (titres seuls)	0
Unités sources tenant dans un chunk de 220	5/230 (2,2 %) — le chunker est le plafond
Plafond honnête du top-5 joint	12,2 % de tronçons complets

Table 8 — Diagnostics de couverture (CI et juge retrieval).

Le diagnostic principal est structurel : seules 5 des 230 unités sources auditées tiennent dans un chunk de 220 tokens : le chunker, et non le ranker, borne la qualité du système.

3.3 Visualisation des distributions

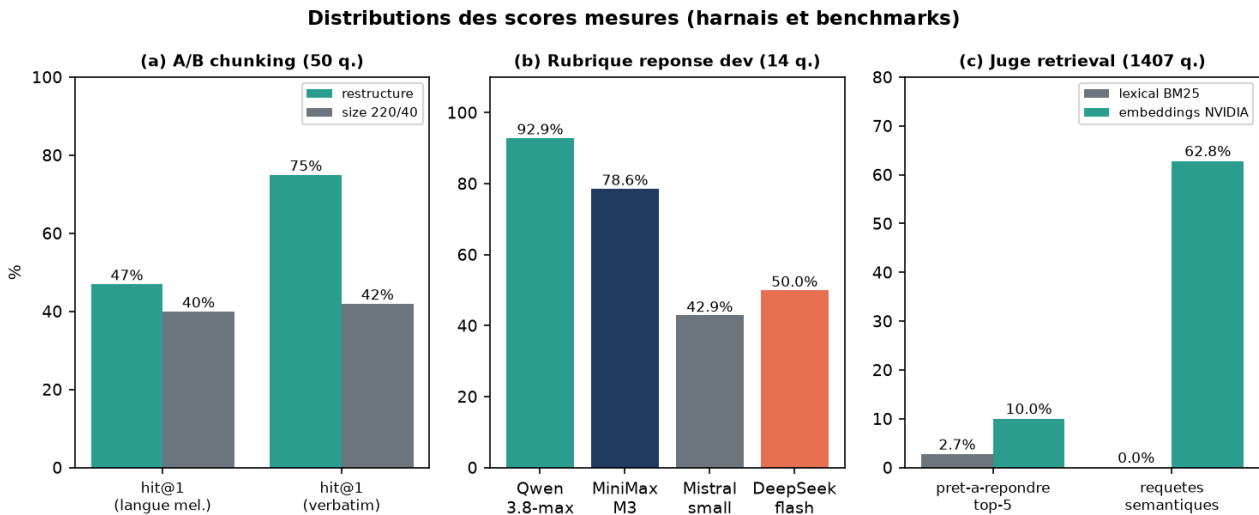


Figure 5 — Distributions des scores mesurés.

Interprétation : (a) restructure bat size sur hit@1 global et verbatim ; (b) Qwen domine la rubrique dev ; (c) les embeddings seuls résolvent les requêtes sans mot commun avec la preuve.

Matrice des métriques retrieval

Lexical vs sémantique : matrice des metriques retrieval (run CI 34005544576, 469 familles / 1407 questions)

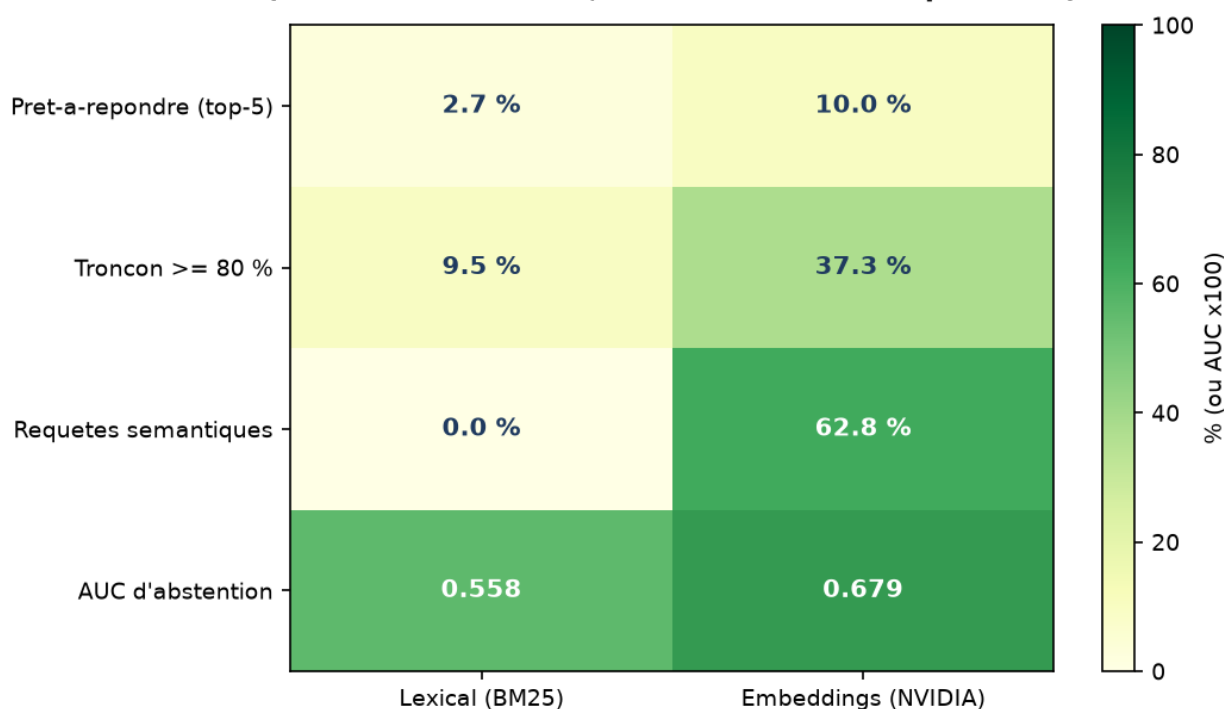


Figure 6 — Lexical vs sémantique (run CI 34005544576).

Interprétation : les embeddings surpassent le lexical sur les quatre axes, avec un écart maximal sur les requêtes sémantiques (62,8 % contre 0 %).

3.4 Comparaison des modèles et évaluation

Banc d'essai des modèles gratuits (développement, 16 cas)

SKU xKiro	Rubrique	Validation	Refus	Temps moy.
qwen/qwen3.8-max:free	13/14 (92,9 %)	16/16	2/2	4,68 s
minimax/minimax-m3:free	11/14 (78,6 %)	15/16	2/2	6,07 s
mistralai/mistral-small-2603	6/14 (42,9 %)	11/16	2/2	8,27 s
deepseek/deepseek-v4-flash	7/14 (50,0 %)	13/16	2/2	7,46 s

Table 9 — Comparaison fraîche des 4 candidats (5 sept. 2026).

Qwen : held-out et sécurité

Contrôle	Résultat
Rubrique réponses	18/18
Faux refus	0/18

Validation citations	18/18
Validation globale	27/27
Refus privé/temps réel	9/9 (garde-fous locaux)
Injections de source	3/3
Temps client moyen	5,25 s (18 appels)

Table 10 — Qwen held-out : 18/18 réponses, 9/9 refus, 3/3 injections.

A/B chunking : restructure contre fenêtre fixe

Mesure	restructure	size 220/40
hit@1 global (50 q., BM25)	47 %	40 %
hit@1 verbatim	75 %	42 %
hit@1 modèles réels	73-76 %	67 %
Gain paraphrase / questions FR	+15 pp / +27 pp	référence
Rappel top-20	inférieur	parfait (100 %)

Table 11 — A/B chunking : harnais-50 et test sur modèles réels (run 34144251576).

Juge de retrieval sans LLM (1407 questions)

Métrique	Lexical (BM25)	Embeddings (NVIDIA)
Tronçon entier dans un chunk du top-5	2,7 %	10,0 %
Tronçon à 80 %	9,5 %	37,3 %
Requêtes sémantiques (145)	0,0 %	62,8 %
AUC d'abstention	0,558	0,679

Table 12 — Juge retrieval : le cross-lingue sans aucun appel génératif.

3.5 Exemple de réponse pour une question

Question	Réponse du système	Statut
What is Murabaha? (en)	1 claim + 1 citation mot-à-mot du guide ; montant repris de la citation.	Livrée
Définition du Salam ? (fr)	1 claim + 1 citation ; formulation valide mais hors rubrique étroite.	Livrée (rubrique 13/14)
Solde de mon compte ? (fr)	Refus local immédiat, zéro appel modèle (question privée).	Refusée

Table 13 — Exemples illustratifs : deux réponses citées, un refus local.

Exemple de reclassement des stratégies de chunking

Stratégie	Chunks	Médiane	Citation entière	R@1 / R@5
size 220/40	310	186	45,4 %	47,2 % / 70,9 %
size 640/40 (gelé)	127	378	58,7 %	63,8 % / 95,0 %
size 420/40	173	354	62,3 %	60,3 % / 89,4 %
manuelle souple	107	557	59,0 %	59,2 % / 86,2 %
manuelle stricte	211	153	59,0 %	55,3 % / 83,0 %

Table 14 — Cinq chunkings mesurés hors-ligne sur 282 questions.

3.6 Grand harnais : état d'avancement

Le grand harnais vise 900 familles de questions plus 100 adversariales. Sont acceptées 469 familles (1 407 paires) de catégorie supported, réparties en 9 shards ; les 200 familles hors-sujet et 50 à preuve insuffisante restent à produire. L'audit croisé indépendant est de 0/469 (second passage par le même modèle). Aucun score de comparaison de réponses n'est gelé : ce harnais est une infrastructure, pas un résultat.

3.7 Conclusion

La réalisation livre un laboratoire complet et mesuré : corpus de 836 chunks à couverture totale, A/B chunking gagné par restructure, Qwen sélectionné sur banc d'essai frais puis confirmé en held-out (18/18), preuve cross-lingue apportée sans LLM (62,8 % contre 0 %) et 133 contrôles hors-ligne au vert. Chaque chiffre est rattaché à son run, son git hash ou son JSON de preuve.

Chapitre 4

Conclusion générale et perspectives

Conclusion

Le stage a permis de concevoir et d'évaluer un système RAG multilingue complet pour documents bancaires : ingestion de 4 documents réels en 836 chunks, indexation par embeddings natifs 2048 d, génération citée avec portillon de citations et refus contrôlés, le tout exposé en CLI, console et microservice REST conteneurisé. Les mesures montrent que le découpage restructuré améliore le hit@1 de 7 points (jusqu'à 9 sur modèles réels), que Qwen 3.8 via xKiro atteint 92,9 % en développement et 100 % en held-out, et que les embeddings résolvent 62,8 % des requêtes sans mot commun contre 0 % au lexical. L'approche a aussi identifié le vrai plafond du système : le chunker, seules 5 unités sources sur 230 tenant dans un chunk de 220 tokens.

Limites et recommandations

- Preuves éparpillées par le chunking fin → passer le défaut applicatif à 420-640 tokens et généraliser l'enrichissement par voisins.
- Serving gratuit non garanti (quotas, routage, identité du modèle) → contrat fournisseur approuvé, capacité mesurée et astreinte avant production.
- Jeux d'évaluation petits et corrélés, audit non indépendant (0/469) → finir les 431 familles, ajouter les 250 négatives, imposer l'audit croisé.
- Pas de garantie bancaire/juridique ni de preuve de sécurité → revue linguistique-métier, tests d'injection élargis, pilote supervisé uniquement.

Perspectives

Trois chantiers prolongent naturellement ce travail : l'enrichissement du contexte (voisins, seuils d'abstention calibrés), l'achèvement du grand harnais trilingue avec gels de versions, et la construction du dossier de production (fournisseur approuvé, revue bancaire indépendante, métriques de disponibilité). Le laboratoire est prêt à les accueillir : chaque piste s'y exprime comme une expérience mesurée, comparée et rejouable en CI.